DOCUMENTO TÉCNICO

Sistema de Gestión de Pólizas

Prueba técnica — Desarrollador TI

Autor: Nelson enrique Guio Rojas

Fecha: 16 de octubre de 2026

Repositorio: https://github.com/neguior/gestion-polizas

Tecnologías: Java 21 · Spring Boot 3.4.5 · Maven

Resumen ejecutivo

Este documento consolida la propuesta de arquitectura, el alcance de la API implementada, las estrategias de optimización de consultas, el procedimiento Git para incorporar una corrección de seguridad y un enfoque de liderazgo técnico para estabilizar un equipo con deuda técnica e incidentes.

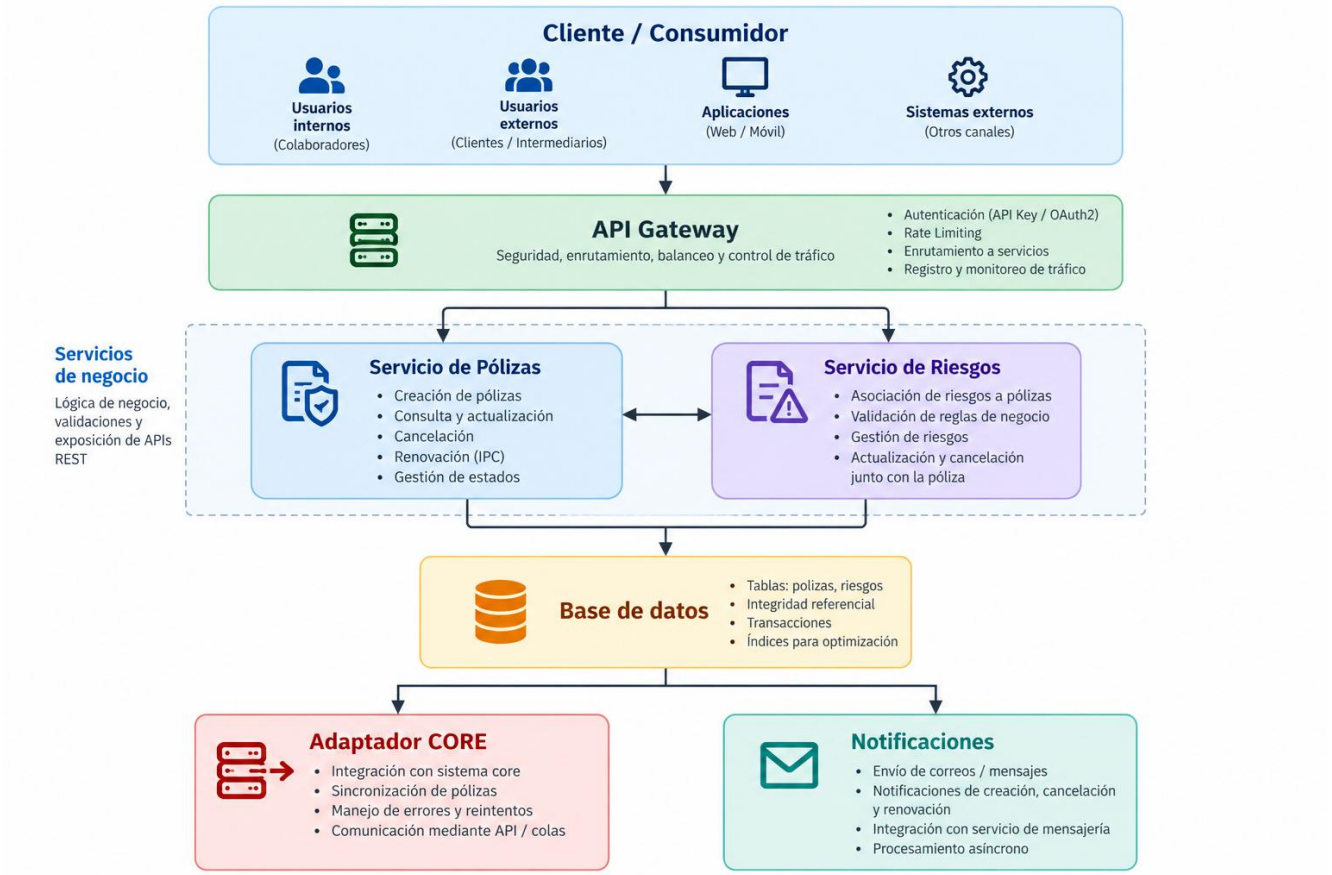
Importante: la arquitectura objetivo y algunas estrategias descritas son propuestas de diseño. La implementación del repositorio es una versión práctica simplificada para demostrar las reglas principales del dominio y los endpoints solicitados.

1. Diseño de arquitectura del sistema

1.1 Componentes y responsabilidades

API Gateway	Punto de entrada; autenticación, límites de tráfico, trazabilidad y enrutamiento.
Servicio de Pólizas	Crear, consultar, renovar y cancelar pólizas; aplicar reglas de negocio.
Servicio de Riesgos	Registrar, consultar y cancelar riesgos asociados a una póliza.
Servicio de Notificaciones	Emitir notificaciones de renovación, cancelación y cambios relevantes.
Adaptador CORE	Aislar la integración con el sistema central y traducir contratos.
Base de datos	Persistir pólizas, riesgos, estados, fechas y datos de auditoría.

1.2 Diagrama lógico



1.3 Patrones de diseño propuestos

Arquitectura hexagonal (Ports and Adapters)

Mantiene el dominio independiente de frameworks y sistemas externos. Los controladores y adaptadores traducen solicitudes; los servicios de aplicación ejecutan casos de uso; los puertos aíslan persistencia y CORE. Facilita pruebas y reemplazo de integraciones.

Arquitectura orientada a eventos

Los cambios relevantes —por ejemplo, póliza renovada o cancelada— pueden publicarse como eventos para que notificaciones y otros consumidores reaccionen sin acoplarse directamente al flujo principal. Para garantizar consistencia se recomienda evaluar el patrón Outbox y consumidores idempotentes.

API Gateway

Centraliza políticas transversales como autenticación, rate limiting, correlación de solicitudes y enrutamiento. No sustituye las validaciones de autorización ni las reglas de negocio dentro de los servicios.

1.4 Modelo de datos conceptual

Póliza	id, tipo, estado, fechaInicio, fechaFin, canon, prima	Una póliza puede tener varios riesgos; la póliza individual admite un solo riesgo.
Riesgo	id, polizaId, descripcion, estado	Cada riesgo pertenece a una póliza.
Evento / auditoría	id, tipo, entidadId, fecha, resultado, correlationId	Permite rastrear operaciones y cambios relevantes.

1.5 Escalabilidad, observabilidad y tolerancia a fallos

- Escalabilidad: mantener servicios sin estado cuando sea posible, ejecutar múltiples instancias detrás de un balanceador y escalar componentes según métricas de carga.
- Datos: definir índices según consultas reales, paginar resultados y revisar planes de ejecución antes de escalar recursos.
- Observabilidad: logs estructurados, métricas de latencia y errores, trazas distribuidas y correlationId propagado entre servicios.
- Resiliencia: timeouts explícitos, reintentos acotados con backoff para fallos transitorios, circuit breaker y operaciones idempotentes.
- Auditoría y seguridad: no registrar secretos ni datos sensibles innecesarios; mantener trazabilidad de acciones críticas.

1.6 Versionado de API

Se propone versionar contratos mediante rutas como /api/v1/polizas. Los cambios incompatibles deben publicarse en una versión mayor, con periodo de transición, documentación y política explícita de retiro de la versión anterior.

2. API REST con Spring Boot

Repositorio de referencia: https://github.com/neguior/gestion-polizas. La aplicación se construyó con Spring Boot 3.4.5 y Maven. La configuración local utiliza H2 en memoria.

2.1 Ejecución local

- Requisitos: JDK compatible con el proyecto y Maven.
- Ejecutar pruebas: mvn clean test
- Iniciar la aplicación: mvn spring-boot:run
- Base URL local: http://localhost:8080
- Autenticación de prueba: incluir el encabezado x-api-key: 123456

2.2 Endpoints principales

GET /polizas	Consulta pólizas con filtros por tipo y estado.
GET /polizas/{id}/riesgos	Lista los riesgos asociados a una póliza.
POST /polizas/{id}/renovar	Renueva la póliza aplicando el incremento IPC configurado y actualiza su estado.
POST /polizas/{id}/cancelar	Cancela la póliza y todos sus riesgos asociados.
POST /polizas/{id}/riesgos	Agrega un riesgo; valida que la póliza sea colectiva.
POST /riesgos/{id}/cancelar	Cancela un riesgo individual.

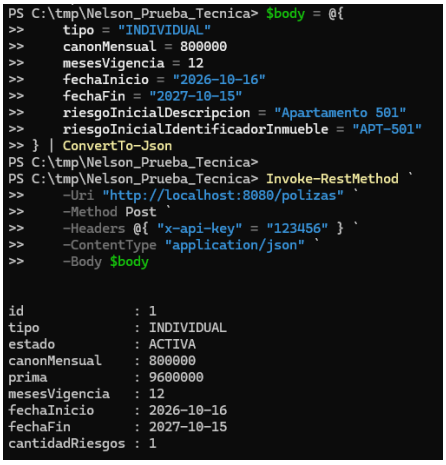
POST /core-mock/evento	Simula la recepción de un evento del CORE y registra el intento en logs.

2.3 Reglas de negocio cubiertas

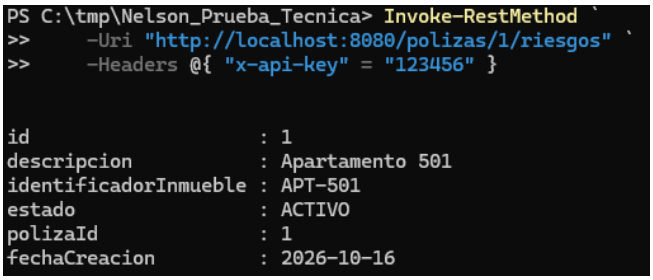
- Una póliza individual admite un único riesgo.
- No se permite renovar una póliza cancelada.
- Al cancelar una póliza, también se cancelan sus riesgos.
- Solo las pólizas colectivas pueden recibir riesgos adicionales.
- Los endpoints protegidos requieren el encabezado x-api-key.

2.4 Validaciones manuales registradas

Crear póliza individual con riesgo inicial (***Figura 1***)	Operación exitosa.
Agregar otro riesgo a póliza individual (***Figura 2***)	HTTP 400.
Cancelar póliza individual (***Figura 3***)	Póliza y riesgo quedaron cancelados.
Crear póliza colectiva y agregar riesgos (***Figura 4***)	Operación exitosa.
Renovar póliza colectiva con IPC de 5,2 % (***Figura 5***)	Canon y prima incrementados; estado RENOVADA.
Renovar póliza cancelada (***Figura 6***)	HTTP 409.
Consultar sin API key (***Figura 7***)	HTTP 401.
Ejecutar mvn clean test (***Figura 8***)	Prueba automatizada ejecutada sin fallos en la última validación registrada.



(*Figura 1*)



```
PS C:\tmp\Nelson_Prueba_Tecnica>
PS C:\tmp\Nelson_Prueba_Tecnica> $bodyRiesgo = @{
>>     descripcion = "Segundo apartamento"
>>     identificadorInmueble = "APT-502"
>> } | ConvertTo-Json
PS C:\tmp\Nelson_Prueba_Tecnica>
PS C:\tmp\Nelson_Prueba_Tecnica> try {
>>     Invoke-RestMethod `
>>         -Uri "http://localhost:8080/polizas/1/riesgos" `
>>         -Method Post `
>>         -Headers @{ "x-api-key" = "123456" } `
>>         -ContentType "application/json" `
>>         -Body $bodyRiesgo
>> } catch {
>>     Write-Host "Código HTTP:" ([int]$_.Exception.Response.StatusCode)
>>     Write-Host "Resultado esperado: 400 Bad Request"
>> }
Código HTTP: 400
Resultado esperado: 400 Bad Request
PS C:\tmp\Nelson_Prueba_Tecnica>
```

(*Figura 2*)

```
PS C:\tmp\Nelson_Prueba_Tecnica> Invoke-RestMethod `
>>     -Uri "http://localhost:8080/polizas/1/cancelar" `
>>     -Method Post `
>>     -Headers @{ "x-api-key" = "123456" }


id              : 1
tipo            : INDIVIDUAL
estado          : CANCELADA
canonMensual    : 800000,00
prima           : 9600000,00
mesesVigencia   : 12
fechaInicio     : 2026-10-16
fechaFin        : 2027-10-15
cantidadRiesgos : 1



PS C:\tmp\Nelson_Prueba_Tecnica>
```

(*Figura 3*)

```
PS C:\tmp\Nelson_Prueba_Tecnica> $bodyColectiva = @{
>>     tipo = "COLECTIVA"
>>     canonMensual = 800000
>>     mesesVigencia = 12
>>     fechaInicio = "2026-10-16"
>>     fechaFin = "2027-10-15"
>> } | ConvertTo-Json
PS C:\tmp\Nelson_Prueba_Tecnica>
PS C:\tmp\Nelson_Prueba_Tecnica> Invoke-RestMethod `
>>     -Uri "http://localhost:8080/polizas" `
>>     -Method Post `
>>     -Headers @{ "x-api-key" = "123456" } `
>>     -ContentType "application/json" `
>>     -Body $bodyColectiva


id              : 2
tipo            : COLECTIVA
estado          : ACTIVA
canonMensual    : 800000
prima           : 9600000
mesesVigencia   : 12
fechaInicio     : 2026-10-16
fechaFin        : 2027-10-15
cantidadRiesgos : 0
```

```
PS C:\tmp\Nelson_Prueba_Tecnica> $riesgo1 = @{
>>     descripcion = "Local comercial 205"
>>     identificadorInmueble = "LOC-205"
>> } | ConvertTo-Json
PS C:\tmp\Nelson_Prueba_Tecnica>
PS C:\tmp\Nelson_Prueba_Tecnica> Invoke-RestMethod `
>>     -Uri "http://localhost:8080/polizas/2/riesgos" `
>>     -Method Post `
>>     -Headers @{ "x-api-key" = "123456" } `
>>     -ContentType "application/json" `
>>     -Body $riesgo1


id                  : 2
descripcion         : Local comercial 205
identificadorInmueble : LOC-205
estado              : ACTIVO
polizaId            : 2
fechaCreacion       : 2026-10-16
```

```
PS C:\tmp\Nelson_Prueba_Tecnica> $riesgo2 = @{
>>     descripcion = "Bodega 12"
>>     identificadorInmueble = "BOD-12"
>> } | ConvertTo-Json
PS C:\tmp\Nelson_Prueba_Tecnica>
PS C:\tmp\Nelson_Prueba_Tecnica> Invoke-RestMethod `
>>     -Uri "http://localhost:8080/polizas/2/riesgos" `
>>     -Method Post `
>>     -Headers @{ "x-api-key" = "123456" } `
>>     -ContentType "application/json" `
>>     -Body $riesgo2


id                  : 3
descripcion         : Bodega 12
identificadorInmueble : BOD-12
estado              : ACTIVO
polizaId            : 2
fechaCreacion       : 2026-10-16
```

(*Figura 4*)

```
PS C:\tmp\Nelson_Prueba_Tecnica>
PS C:\tmp\Nelson_Prueba_Tecnica> Invoke-RestMethod `
>>     -Uri "http://localhost:8080/polizas/2/renovar" `
>>     -Method Post `
>>     -Headers @{ "x-api-key" = "123456" }


id              : 2
tipo            : COLECTIVA
estado          : RENOVADA
canonMensual    : 841600,00
prima           : 10099200,00
mesesVigencia   : 12
fechaInicio     : 2027-10-16
fechaFin        : 2028-10-15
cantidadRiesgos : 2
```

(*Figura 5*)

```
PS C:\tmp\Nelson_Prueba_Tecnica> try {
>>     Invoke-RestMethod `
>>         -Uri "http://localhost:8080/polizas/1/renovar" `
>>         -Method Post `
>>         -Headers @{ "x-api-key" = "123456" }
>> }
>> catch {
>>     Write-Host "Código HTTP:" ([int]$_.Exception.Response.StatusCode)
>>     Write-Host "Respuesta:" $_.ErrorDetails.Message
>> }
Código HTTP: 409
Respuesta:
PS C:\tmp\Nelson_Prueba_Tecnica>
```

(*Figura 6*)

```
PS C:\Users\negui>
PS C:\Users\negui> try {
>>     Invoke-RestMethod `
>>         -Uri "http://localhost:8080/polizas" `
>>         -Method Get
>> }
>> catch {
>>     Write-Host "Código HTTP:" ([int]$_.Exception.Response.StatusCode)
>>     Write-Host $_.ErrorDetails.Message
>> }
Código HTTP: 401
{"error":"API key ausente o inv♦lida"}
PS C:\Users\negui>
```

(*Figura 7*)

```
WARNING: A Java agent has been loaded dynamically (C:\Users\negui\.m2\repository\net\bytebuddy\byte-buddy-agent\1.15.11\
byte-buddy-agent-1.15.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 7.358 s -- in co.com.segurosbolivar.polizas.Gesti
onPolizasApplicationTests
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  12.081 s
[INFO] Finished at: 2026-10-16T15:41:26-05:00
[INFO] ------------------------------------------------------------------------
PS C:\tmp\Nelson_Prueba_Tecnica>
PS C:\tmp\Nelson_Prueba_Tecnica>
PS C:\tmp\Nelson_Prueba_Tecnica>
PS C:\tmp\Nelson_Prueba_Tecnica> Get-Content .\src\main\java\co\com\segurosbolivar\polizas\model\TipoPoliza.java
package co.com.segurosbolivar.polizas.model;

public enum TipoPoliza {
    INDIVIDUAL, COLECTIVA
}
PS C:\tmp\Nelson_Prueba_Tecnica>
PS C:\tmp\Nelson_Prueba_Tecnica>
```

(*Figura 8*)

2.5 Límites conocidos y mejoras

- H2 está configurada en memoria: los datos se reinician al detener la aplicación. Para persistencia real, usar una base de datos externa.
- El endpoint /core-mock/evento es una simulación; no realiza integración real con el CORE.
- La API key es un mecanismo de demostración. En producción debe sustituirse por una solución de identidad y autorización robusta, gestión segura de secretos y rotación de credenciales.
- Como siguientes pasos: ampliar pruebas unitarias e integración, documentar OpenAPI, añadir migraciones de esquema, auditoría y manejo uniforme de errores.

3. Optimización de consulta SQL para alto volumen

Escenario: consultar 10 millones de órdenes relacionadas con 500.000 clientes y filtrar clientes cuyo país sea México.

3.1 Estrategias recomendadas

Índices alineados con filtros y joins

Evaluar índices en clientes(pais, id) y órdenes(cliente_id), ajustándolos al motor y a la selectividad real. Evitar índices redundantes.

Filtrar temprano

Reducir primero el conjunto de clientes del país requerido y unirlo con órdenes. El optimizador puede reordenar el plan, por lo que debe verificarse con EXPLAIN / plan de ejecución.

Seleccionar solo columnas necesarias

Evitar SELECT *; reduce lectura, transferencia y presión de memoria.

Particionamiento y estrategia física

Si las consultas y el crecimiento lo justifican, evaluar particionar órdenes por fecha u otra clave de acceso. El particionamiento debe responder al patrón de consultas.

Estadísticas y medición

Mantener estadísticas actualizadas, revisar cardinalidad, lecturas, tiempo, uso de memoria y spills; comparar planes antes y después.

3.2 Ejemplo SQL

```
SELECT o.id, o.cliente_id, o.fecha, o.total
FROM ordenes AS o
JOIN clientes AS c ON c.id = o.cliente_id
WHERE c.pais = 'MEXICO';
```

La consulta es ilustrativa. La decisión final debe basarse en el motor de base de datos, la distribución de datos, los índices existentes y el plan de ejecución real.

4. Git: incorporar una corrección de seguridad

Objetivo: llevar únicamente el commit que corrige una vulnerabilidad desde main a feature/new-login, sin incorporar los demás cambios de main.

1. Actualizar referencias remotas: git fetch origin
2. Cambiar a la rama destino: git switch feature/new-login
3. Identificar el commit de seguridad: git log origin/main --oneline
4. Aplicar solo el commit seleccionado: git cherry-pick <hash-del-commit>
5. Resolver conflictos si aparecen, revisar el diff y ejecutar pruebas.
6. Confirmar el estado: git status; luego publicar la rama si corresponde: git push origin feature/new-login

Si hay conflictos: editar los archivos, ejecutar git add <archivos> y git cherry-pick --continue. Para abortar la operación: git cherry-pick --abort. Antes de aplicar, confirmar que el commit es autocontenido y no depende de otros commits que no estarán en la rama destino.

5. Liderazgo técnico y estabilización del equipo

Contexto: equipo de ocho personas, deuda técnica cercana al 40 % en servicios clave, diez incidentes críticos en el último mes, entrega comprometida en tres semanas, estándares poco consistentes y dos desarrolladores junior.

5.1 Prioridades para las primeras dos semanas

- Días 1–3: revisar incidentes, métricas, arquitectura y backlog; identificar causas recurrentes y riesgos de la entrega.
- Días 4–5: acordar con negocio qué es imprescindible para la fecha y qué puede aplazarse; definir criterios de aceptación.
- Semana 2: corregir primero las causas de incidentes de mayor impacto, añadir pruebas de regresión y reducir riesgos en los componentes críticos.
- Mantener un registro visible de decisiones, riesgos, responsables y fechas de seguimiento.

5.2 Organización del trabajo

- Asignar responsables claros por servicio o componente y evitar dependencias innecesarias.
- Emparejar perfiles sénior con los dos desarrolladores junior en tareas acotadas, con revisión y retroalimentación frecuentes.
- Reservar capacidad explícita para estabilización y deuda técnica, sin perder de vista el compromiso de negocio.
- Realizar seguimiento diario breve a bloqueos, incidentes, calidad y alcance de la entrega.

5.3 Métricas de seguimiento

Confiabilidad	Incidentes críticos, tasa de errores, disponibilidad, MTTR.
Entrega	Frecuencia de despliegue, lead time, cumplimiento de hitos.
Calidad	Defectos escapados, cobertura útil de pruebas, fallos de pipeline.
Deuda técnica	Riesgos priorizados, componentes con pruebas, tiempo dedicado a remediación.

Equipo	Bloqueos, distribución de carga, evolución de autonomía y revisiones.

5.4 Prácticas mínimas no negociables

- Pull request con revisión por pares y checklist de seguridad y calidad.
- Pruebas automatizadas para reglas críticas y regresiones asociadas a incidentes.
- Pipeline CI con compilación, pruebas y análisis estático antes de integrar.
- Gestión segura de secretos, permisos mínimos y registro de decisiones técnicas relevantes.
- Definición de terminado que incluya pruebas, observabilidad y documentación mínima.

5.5 Manejo de la presión

Comunicar con transparencia el alcance, riesgos y opciones; negociar prioridades con datos; evitar prometer fechas sin validar capacidad; proteger al equipo de cambios no priorizados y escalar temprano cualquier riesgo que amenace la entrega o la estabilidad del servicio.

Conclusión

La solución combina una implementación funcional de reglas esenciales con una propuesta de evolución hacia una arquitectura observable, resiliente y escalable. La prioridad inmediata es demostrar el comportamiento solicitado, explicar las decisiones con claridad y reconocer de forma explícita qué está implementado y qué queda como mejora.

Referencias de trabajo

- Repositorio del proyecto: https://github.com/neguior/gestion-polizas
- Ejecución de pruebas: mvn clean test
- Inicio local: mvn spring-boot:run